

Prepárate para el mercado laboral

Desarrollo de un Portafolio Digital

Proyecto integrador — Empleabilidad en la Industria Digital

Sandra Castillo

Parte 1 — Investigación de empresa

Empresa seleccionada: Rayen Salud

Empresa chilena de servicios tecnológicos en salud, con más de 15 años de trayectoria en Chile y Latinoamérica, dedicada a la Transformación Digital en Salud.

Por qué Rayen Salud (perspectiva técnica)

Criterio	Lo que ofrece Rayen Salud
Valores y propósito	Contribuir a la Transformación Digital en Salud de Chile y Latinoamérica; co-creación de soluciones bajo estándares internacionales de seguridad de la información.
Tipo de productos	Soluciones tecnológicas para el sector salud: gestión clínica y de procesos hospitalarios y de centros médicos, implementadas en Chile y la región.
Tecnologías que utilizan	No publican su stack específico, pero exigen bases sólidas en desarrollo backend, bases de datos relacionales y buenas prácticas de arquitectura — perfil técnico coherente con Java EE / Spring Boot.
Metodologías de trabajo	Metodologías ágiles de co-creación con clientes del sector salud, equipos colaborativos.
Enfoque en innovación	15+ años desarrollando e implementando soluciones innovadoras en salud digital, con foco regional (Chile y Latam).

3 formas concretas de aportar valor

- Experiencia reciente aplicando patrón MVC, DAO y persistencia con JDBC/JPA — arquitectura en capas directamente relevante para software de gestión clínica que debe ser mantenible y auditable.
- Conocimiento de control de sesiones y roles de usuario (admin/usuario), clave en salud, donde el acceso a información de pacientes debe estar bien segmentado.
- Capacidad de seguir consignas técnicas rigurosas y adaptarme a bases de datos distintas (migración de H2 a MariaDB sin perder el core del proyecto), señal de flexibilidad ante estándares de un cliente de salud.

3 preguntas para el reclutador

- Al trabajar "co-creando" soluciones con clientes del sector salud, ¿cómo participa alguien junior en ese proceso de levantamiento de requerimientos?
- ¿Qué estándares de seguridad de la información aplican al código desde el día uno para alguien que recién se integra?
- ¿Cómo es la transición de la práctica profesional a un rol permanente dentro del equipo de desarrollo?

Parte 2 — Portafolio técnico

Portafolio (LinkedIn): [linkedin.com/in/kavrita-castillo-101765410](https://www.linkedin.com/in/kavrita-castillo-101765410)

Proyectos incluidos

- Biblioteca Digital UNTEC — Java EE (JSP, Servlets, MVC, DAO, Tomcat, MariaDB)
- ZooManager — Spring Boot (MVC, JPA, Spring Security, REST)
- SmartTask — Java / POO (herencia, polimorfismo, interfaces, colecciones, JUnit 5, Maven) · github.com/kavrita/mismarttask

Caso de estudio — Biblioteca Digital UNTEC

Breve descripción de la actividad

Desarrollo completo de una aplicación web de gestión de biblioteca digital con Java EE, cubriendo desde la estructura inicial del proyecto hasta el despliegue: gestión de libros, préstamos y sesiones de usuario con distintos niveles de acceso (admin/usuario), construida con JSP, Servlets, patrón MVC y capa DAO sobre JDBC.

Desafío principal

El entorno de desarrollo pedido originalmente (Eclipse EE + H2) no calzaba con el flujo de trabajo real, por lo que fue necesario adaptar todo el proyecto a Visual Studio Code con Tomcat 9 y MariaDB, resolviendo problemas de compatibilidad de versiones (javax.* vs jakarta.*) y de conexión JDBC — incluyendo un conflicto de autenticación de MariaDB con el usuario root que obligó a crear un usuario de base de datos dedicado.

Solución propuesta

Se reestructuró la configuración del proyecto para Tomcat 9 manteniendo el patrón MVC y DAO intactos, y se diagnosticó un bug crítico donde los formularios JSP se corrompían silenciosamente (apuntaban a rutas .html en vez de a los Servlets, con una condición de admin hardcoded), agregando logs de depuración temporales en los Servlets de login y de gestión de libros para aislar el origen del problema antes de reemplazar los archivos afectados.

Herramienta técnica utilizada

Java EE (JSP, Servlets), patrón MVC, DAO con JDBC, JSTL, Apache Tomcat 9, MariaDB, Visual Studio Code.

Principales aprendizajes

- La importancia de no asumir que el entorno "sugerido" en una consigna es la única opción viable: adaptar la arquitectura a otro stack de despliegue sin romper el patrón de diseño.
- Debugging metódico en un entorno multicapa (vista JSP → Servlet → DAO → base de datos), usando logging temporal como herramienta de diagnóstico en vez de adivinar la causa.
- Gestión de conflictos de entorno reales (autenticación de base de datos, versiones de Tomcat) que no aparecen en la teoría pero sí en el día a día de un desarrollador.

Métricas de impacto logradas

- 100% de las 6 lecciones del módulo cubiertas (estructura inicial, JSP, Servlets, DAO, MVC, despliegue).
- Migración completa de stack de desarrollo (Eclipse/H2 → VS Code/MariaDB) sin pérdida de funcionalidad.
- Bug crítico de login y de creación de libros como admin, diagnosticado y resuelto en una sola sesión de depuración dirigida.

Habilidades técnicas aplicadas

Java EE, arquitectura MVC, patrón DAO, JDBC, gestión de sesiones de usuario, SQL/MariaDB, debugging en aplicaciones multicapa, despliegue en servidores de aplicaciones (Tomcat, .WAR).

Justificación de la elección

Se eligió este proyecto porque no fue un ejercicio de libro de texto: implicó resolver problemas reales de entorno y un bug que no era evidente a primera vista, lo que refleja mejor cómo se trabaja en un equipo de desarrollo real — no solo seguir instrucciones, sino diagnosticar y adaptar.

Caso de estudio — ZooManager (Spring Boot)

Breve descripción de la actividad

Desarrollo de una aplicación Spring Boot siguiendo la guía de evaluación del módulo (Maven, Spring MVC, JPA, Spring Security, REST), pero reemplazando el dominio académico propuesto originalmente (gestión de estudiantes/cursos) por uno propio: gestión de un zoológico (entidades Animal y Hábitat), manteniendo los mismos requisitos técnicos de la consigna.

Desafío principal

El mayor desafío fue adaptar toda la lógica y las relaciones entre entidades del dominio académico original (estudiantes, cursos) al nuevo dominio de zoológico (animales, hábitats), sin una guía paso a paso para ese caso específico. La capa REST fue la parte que más costó comprender en profundidad, más allá de hacerla funcionar.

Solución propuesta

Se optó por un enfoque de prueba y error guiado: modificar el código, romperlo, y usar los errores de compilación como guía para entender cómo se relacionaban realmente las clases, los repositorios JPA y los controladores, en vez de memorizar la solución de un dominio ajeno. Este método ("romper código para aprender", como lo plantea el profesor del curso) permitió ir ajustando el modelo Animal/Hábitat hasta que la aplicación compilaba y funcionaba de forma coherente con los requisitos de Spring Security y JPA de la guía.

Herramienta técnica utilizada

Spring Boot, Spring MVC, Spring Data JPA, Spring Security, Maven, arquitectura REST.

Principales aprendizajes

- Cambiar el dominio de una consigna sin perder los requisitos técnicos exige entender la arquitectura a fondo, no solo copiar una estructura.
- Leer con atención los errores de compilación es una herramienta de aprendizaje tan válida como leer documentación.
- La capa REST es un área identificada para seguir reforzando — reconocerlo es el primer paso para profundizar en ella.

Métricas de impacto logradas

- Migración completa del dominio (estudiantes/cursos → animales/hábitats) manteniendo Maven, MVC, JPA, Spring Security y REST según la guía del módulo.

Habilidades técnicas aplicadas

Spring Boot, Spring MVC, JPA, Spring Security, diseño de API REST, Maven, debugging por lectura de errores de compilación.

Justificación de la elección

Se incluye este proyecto porque muestra la capacidad de adaptar una arquitectura a un dominio propio y distinto al pedido, además de honestidad sobre un área técnica (REST) que aún está en proceso de consolidación — algo valorable en un perfil junior que reconoce sus propios límites y sigue aprendiendo.

Caso de estudio — SmartTask (Java / POO)

Breve descripción de la actividad

Desarrollo de una aplicación de consola en Java para gestión de tareas personales: agregar tareas normales y urgentes, listar tareas activas y completadas, marcar tareas como completadas, eliminarlas y asignarles IDs automáticos. Construida aplicando programación orientada a objetos (herencia, polimorfismo, interfaces, colecciones), con pruebas unitarias en JUnit 5 y build gestionado con Maven.

Desafío principal

El manejo de colecciones para organizar y filtrar las tareas (activas, completadas, urgentes) fue el punto más difícil del diseño. A esto se sumaron las pruebas unitarias con JUnit 5, cuya lógica en código costó más de entender que el resto del proyecto.

Solución propuesta

Al igual que en ZooManager, se resolvió con un enfoque de romper el código intencionalmente para entender por qué fallaba y así aprender el comportamiento real de las colecciones y de las pruebas JUnit, en lugar de solo seguir un ejemplo sin comprenderlo del todo.

Herramienta técnica utilizada

Java, POO (herencia, polimorfismo, interfaces), colecciones, JUnit 5, Maven.

Principales aprendizajes

- Practicar con colecciones en un caso concreto (tareas activas/completadas/urgent) ayuda a fijar conceptos que en teoría cuestan más.
- Las pruebas unitarias con JUnit 5 son un área a seguir reforzando, especialmente en cómo se estructuran en código.
- El método de "romper código para aprender" es aplicable a distintos proyectos, no solo a uno puntual.

Métricas de impacto logradas

- Aplicación funcional con operaciones CRUD completas sobre tareas (agregar, listar, completar, eliminar) e IDs automáticos.

Habilidades técnicas aplicadas

Java, POO (herencia, polimorfismo, interfaces), manejo de colecciones, pruebas unitarias con JUnit 5, Maven.

Justificación de la elección

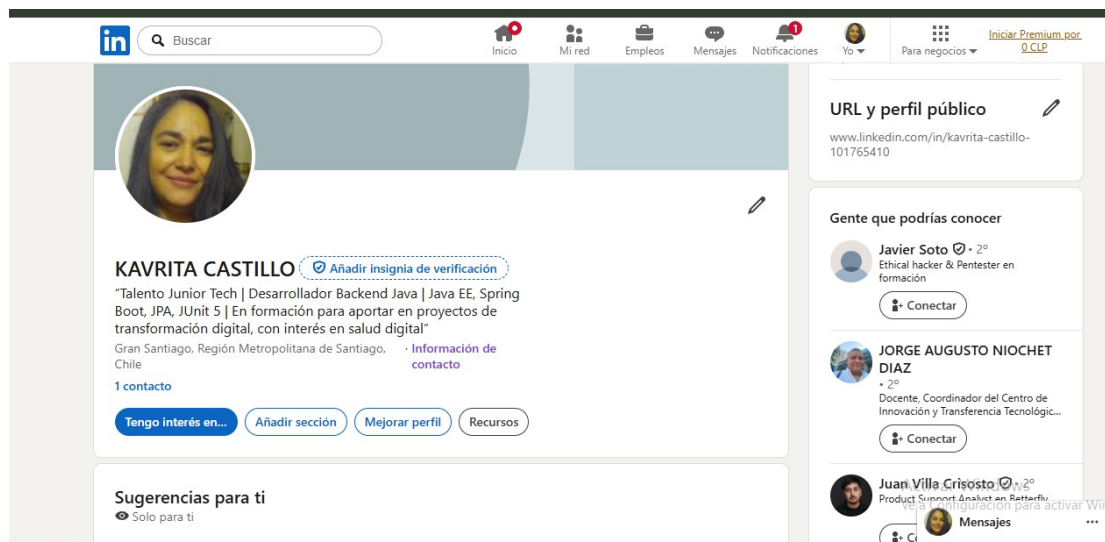
Se incluye este proyecto porque complementa el perfil con un ejemplo de POO pura en Java (sin framework), y porque, junto con ZooManager, refleja de forma consistente un método de aprendizaje propio: enfrentar el error directamente para entender la causa, en vez de evitarlo.

Parte 2: Desarrollo de Empleabilidad

Perfil de LinkedIn optimizado

Link del perfil: [linkedin.com/in/kavrita-castillo-101765410](https://www.linkedin.com/in/kavrita-castillo-101765410)

[Agregar aquí la captura de pantalla del perfil de LinkedIn actualizado, junto con una breve descripción de los cambios aplicados: título con palabras clave, foto de perfil profesional, banner relacionado a tecnología, y sección "Acerca de" actualizada.]



Cambios aplicados: actualicé el título de mi perfil para incluir palabras clave relacionadas a mi especialidad (Talento Junior Tech, desarrollo backend Java, tecnologías como Java EE, Spring Boot, JPA y JUnit 5) y mi interés en salud digital, con el objetivo de dar mayor visibilidad ante reclutadores que busquen ese perfil técnico.

Extracto profesional — "Acerca de"

Soy Sandra Castillo, actualmente en formación como Talento Junior Tech, especializándome en desarrollo backend con Java. Mi recorrido combina Java EE clásico (JSP, Servlets, patrón MVC y DAO) con Spring Boot moderno (JPA, Spring Security, REST), lo que me permite entender tanto los fundamentos como las herramientas actuales del ecosistema Java.

Mi objetivo es insertarme en un equipo de desarrollo donde pueda seguir creciendo técnicamente y aportar desde el primer día, idealmente en un sector donde la tecnología tenga un impacto concreto — como la salud digital, un área que me interesa especialmente por el compromiso que exige con la calidad y seguridad del software.

Entre mis proyectos destacados está Biblioteca Digital UNTEC, una aplicación Java EE de gestión de biblioteca donde tuve que migrar todo el entorno de desarrollo original y diagnosticar un bug crítico en el flujo de login; también desarrollé ZooManager con Spring Boot y SmartTask, una app de consola en Java aplicando programación orientada a objetos y pruebas con JUnit 5.

Manejo Java EE, Spring Boot, JPA, patrón MVC/DAO, SQL, JUnit 5 y Maven, y sigo reforzando activamente el diseño de APIs REST y las pruebas unitarias en profundidad.

Como forma de trabajar, mi mayor fortaleza es el debugging metódico: cuando algo no funciona, prefiero entender la causa real antes que aplicar una solución rápida, incluso si eso significa "romper" el código a propósito para aprender cómo funciona de verdad. Eso viene acompañado de adaptabilidad — he tenido que migrar stacks completos de un entorno a otro sin perder el diseño original.

Estoy buscando mi primera oportunidad como Junior Developer o Trainee en un equipo donde pueda seguir aprendiendo con acompañamiento, aportar con responsabilidad desde el inicio y crecer hacia un rol backend sólido.

Elevator Pitch

"Hola, soy Sandra Castillo, y estoy terminando mi formación como Talento Junior Tech, enfocada en desarrollo backend con Java. He trabajado con Java EE y Spring Boot en proyectos reales de curso, como una aplicación de gestión de biblioteca donde tuve que migrar todo el entorno de desarrollo y resolver un bug crítico de login bajo presión de tiempo.

Lo que me distingue es mi forma de resolver problemas: no me conformo con que algo funcione, necesito entender por qué funciona — por eso mi método de aprendizaje es literalmente romper el código para descubrir cómo se comporta de verdad. Eso me ha dado una base sólida en debugging y adaptabilidad a distintos entornos técnicos.

Estoy buscando mi primera oportunidad como developer junior en un equipo donde pueda aportar desde el día uno y seguir creciendo, idealmente en un sector como la salud digital, donde la tecnología realmente importa. Sería un gusto conversar sobre cómo puedo aportar a su equipo."

PLUS — Propuesta de publicación profesional

Título / frase de entrada

"Rompi mi propio código a propósito. Así aprendí más que en cualquier tutorial."

Cuerpo del texto

Hace poco terminé de desarrollar una aplicación de gestión de biblioteca con Java EE. Todo iba bien hasta que me topé con un bug que no tenía ningún sentido: el login y la creación de libros como administrador simplemente no funcionaban, y no había ningún error visible que me dijera por qué.

En vez de buscar la respuesta directa, decidí hacer algo que me han insistido varios formadores: romper el código a propósito. Agregué logs de depuración en cada capa —vista, servlet, DAO, base de datos— y fui aislando el problema paso a paso, hasta descubrir que los formularios apuntaban a rutas equivocadas, con una condición de administrador mal configurada desde el inicio.

Ese proceso me enseñó algo que ningún curso teórico me había dado: no hay que asumir que el camino "sugerido" es el único correcto, y entender un error a fondo vale más que resolverlo rápido. Ese mismo método lo he aplicado después en otros proyectos, incluyendo uno con Spring Boot y otro en Java puro con pruebas unitarias — y cada vez que "rompo" algo a propósito, termino entendiendo mejor cómo funciona de verdad.

Sigo en formación, sigo aprendiendo, y sigo prefiriendo entender el porqué antes que memorizar el cómo.

Hashtags

#TalentoJuniorTech #DesarrolloBackend #JavaEE #AprendizajeContinuo #MiPrimerTrabajoIT

Plus — Matriz FODA personal

Interno

Fortalezas	Áreas de oportunidad
Capacidad de debugging metódico en apps multicapa (demostrado con el bug de login en Biblioteca UNTEC)	Profundizar en el diseño de APIs REST (buenas prácticas, no solo hacerlas funcionar) — identificado como punto débil en ZooManager
Adaptabilidad a cambios de stack/entorno sin romper el diseño (migración Eclipse/H2 → VS Code/MariaDB)	Ganar experiencia en despliegue en entornos cloud (AWS, Azure) más allá de servidores locales
Base sólida en POO aplicada (herencia, polimorfismo, interfaces — SmartTask)	Avanzar hacia patrones de diseño más avanzados, más allá de los fundamentos de POO
Conocimiento de dos ecosistemas backend Java (Java EE clásico y Spring Boot moderno)	Incorporar herramientas de IA aplicadas al desarrollo, dado el valor que el mercado les da actualmente
Testing con JUnit 5 como práctica ya incorporada	Profundizar la lógica interna de las pruebas unitarias (JUnit 5), no solo aplicarlas siguiendo ejemplos — dificultad identificada en SmartTask

Externo

Amenazas	Oportunidades
Desempleo general en Chile en su punto más alto en 5 años (9,1% trimestre feb-abr 2026), mercado saturado de egresados	Alta demanda específica en transformación digital, IA y talento técnico especializado, incluso en un mercado que se contrae
Brecha entre lo que forman universidades/bootcamps y lo que piden las empresas en IA	Empresas retomando contratación al confirmar que la IA no reemplaza coordinación de equipos ni toma de decisiones complejas
Roles junior "tradicionales" achicándose: automatización cubre tareas rutinarias y repetitivas	Roles junior rediseñados hacia validación/supervisión de código generado por IA y testing avanzado
Competencia salarial: perfiles con IA+datos+negocio ganan sueldos premium frente a perfiles solo técnicos	Sector salud/healthtech (como Rayen Salud) con foco regional fuerte y crecimiento sostenido en Chile y Latam
Presión por dominar múltiples stacks a la vez (backend, cloud, IA)	Juniors que ya manejan herramientas de IA ganan 10–15% más que sus pares — habilidad sumable ahora, en formación